



## ***BAHIR DAR INSTITUTE OF TECHNOLOGY***

FACULTY OF COMPUTING

SOFTWARE ENGINEERING

INDIVIDUAL ASSIGNMENT

OPERATING SYSTEM

***NAME :Bezawit Simenew***

***Id: 1701702***

***SECTION: B***



Edit with WPS Office



Edit with WPS Office

# Project Work: Operating System and System Programming

## 1. Installation of Fuchsia OS in Virtual Environment.

### A. Introduction (Background and Motivation)

Fuchsia OS is a modern operating system developed by Google. It is different from traditional operating systems like Windows or Linux because it is built using a new design approach called a microkernel, known as Zircon. This new design makes the system more flexible, faster, and easier to manage compared to older operating systems. Unlike android or chromeOS, it is not based on Linux.

Today, technology is growing very fast, and people use many different devices such as smartphones, laptops, tablets, and smart devices. Because of this, there is a need for an operating system that can work smoothly on all these devices. Traditional operating systems sometimes face problems like slow updates, security issues, and difficulty working across different platforms. Fuchsia OS is being developed to solve these problems by providing a system that is more secure, efficient, and adaptable.

One of the main features of Fuchsia OS is its strong focus on security. Nowadays, cyber attacks and data breaches are increasing, so it is very important for operating systems to protect user data. Fuchsia OS is designed with built-in security features that help keep the system safe.

*For example:* it controls how applications access system resources and ensures that one application cannot easily affect another. This makes the system more reliable and secure.



Another reason for choosing Fuchsia OS for this project is to understand how modern operating systems are evolving. Unlike older systems, Fuchsia is still under development, which makes it interesting to study. By working on this project, we can learn new concepts and technologies that may be used in future operating systems. It also helps us improve our practical skills in system installation and configuration.

In this project, Fuchsia OS is installed and run using a virtual environment instead of installing it directly on the computer. This is because Fuchsia OS does not yet support direct installation on most devices. Virtualization allows us to create a virtual computer inside our system where we can safely run and test the operating system. This method is very useful because it does not affect the main operating system and is widely used for learning and testing purposes.

Overall, Fuchsia OS represents a new direction in operating system development. It focuses on better performance, improved security, and the ability to work across different devices. This project helps us understand its basic concepts and gives us hands-on experience with modern tools and technologies.

An operating system (OS) is the core software that manages computer hardware and software resources. Modern systems require faster, secure, and scalable operating systems.

Fuchsia OS is a new open-source operating system developed by Google. Unlike traditional systems such as Linux-based Android or Windows, Fuchsia is built on a microkernel called Zircon. This makes it more secure, flexible, and suitable for modern devices like smartphones, IoT devices, and embedded systems.

The motivation of this project is to explore a modern operating system, understand its architecture, and learn how to install and run it in a virtual environment.



## B.Objectives

The main goal of this project is to understand modern operating systems by working with Fuchsia OS, which is being developed by Google. Instead of only learning theory, this project focuses on gaining practical experience by actually installing and running the system in a virtual environment.

One of the important objectives is to learn how to install an operating system using virtualization. Since Fuchsia OS cannot be directly installed on most computers, using a virtual environment helps us run it safely without affecting our main system. This also helps us understand how virtual machines work and why they are useful in real-life situations, especially for testing and learning new technologies.

Another objective is to understand the basic structure and features of Fuchsia OS. It is different from traditional operating systems because it uses a modern design. By exploring it, we can learn how new operating systems are being developed to be faster, more secure, and more efficient.

This project also aims to improve practical skills. While installing the system, we use different commands and tools, which helps us become more familiar with technical processes. If any problems occur during installation, solving them helps us build problem-solving skills and confidence.

In addition, the project helps us understand the importance of system security and performance. By observing how Fuchsia OS works, we can see how modern systems are designed to protect data and manage resources efficiently.

Finally, this project encourages self-learning. Since Fuchsia OS is still new and not widely used, we may need to explore and learn on our own. This helps us become more independent learners and prepares us for future technologies.



Overall, the objective of this project is:

- To understand the concept of modern operating systems
- To study the architecture of Fuchsia OS
- To install Fuchsia OS in a virtual environment
- To analyze its features and performance
- To identify challenges during installation and usage

## C.Requirements

To successfully install and run Fuchsia OS, it is important to prepare both hardware and software requirements properly. These requirements ensure that the system runs smoothly and the installation process is completed without major problems.

### 1. Hardware Requirements

The hardware requirements focus on the physical components of the computer. Since Fuchsia OS is run inside a virtual environment, the system needs to handle both the main operating system and the virtual system at the same time.

1. RAM (Memory): is very important. A minimum of 8 GB RAM is recommended. This is because virtualization consumes a lot of memory. If the RAM is too small, the system may become very slow or even fail to run the emulator properly. More RAM (like 16 GB) will give better performance.

2. Processor(CPU) : the computer must have a 64-bit processor (CPU). Modern operating systems and virtualization tools require a 64-bit architecture to work efficiently. Also, the processor should support virtualization technology (such as Intel VT-x or AMD-V). This feature allows



the computer to create and run virtual machines more effectively.

3. storage space : At least 20 GB of free disk space is needed. This is because Fuchsia OS requires downloading large files, building the system, and storing temporary data during installation. If there is not enough space, the installation may stop in the middle.

Additionally, having a good processor speed (for example, Intel Core i5 or above) is helpful. Building Fuchsia OS takes time, and a faster processor will reduce the waiting time significantly.

## 2. Software Requirements

The software requirements include the programs and tools needed to install and run Fuchsia OS.

1. Windows 10 or Windows 11 : This acts as the main environment where all installation steps will be performed.

2. Git : Git is used to download (clone) the Fuchsia OS source code from the internet. Without Git, you cannot access the project files needed for installation.

3. Python : Python is used to run setup scripts that prepare the system for building Fuchsia OS. These scripts automate many tasks and make the installation easier.

4. QEMU: which is a virtualization tool. QEMU creates a virtual machine where Fuchsia OS runs like a real computer. It allows you to test the operating system safely without affecting your main system. In many cases, QEMU is automatically managed during the setup process.



In addition, a stable internet connection is required. This is because downloading the Fuchsia OS files and tools depends on internet access. A slow or unstable connection can cause delays or errors during installation.

Finally, basic knowledge of using Command Prompt or PowerShell is important. Many steps in the installation process require typing commands, so being familiar with these tools will help you complete the process more easily.

Overall

By preparing both the hardware and software requirements properly, the installation of Fuchsia OS becomes much easier and more successful. It also helps avoid common errors.

## D. Installation step

In this project, Fuchsia OS is not installed directly on the computer. Instead, it is installed and run inside a virtual environment. This makes the process safer and easier because it does not affect the main system.

The installation was carried out step by step as follows:

**First**, the necessary tools were prepared. This includes installing Git and Python on the computer. These tools are important because they help in downloading and setting up the Fuchsia OS files. After installing them, the Command Prompt (or PowerShell) was opened to start the process.

**Next**, the Fuchsia OS source code was downloaded from its official repository. This was done using Git with a simple command. The download process took some time depending on the internet speed because the file size is large. After downloading, the system was moved into the Fuchsia directory to continue the setup.



Edit with WPS Office



***After that***, the environment was prepared by running a setup script. This step automatically installs all the required dependencies and tools needed to build Fuchsia OS. It simplifies the process by handling many configurations in the background.

Then, the system build configuration was set. In this project, a virtual machine setup using QEMU was selected. This configuration tells the system how Fuchsia OS should be built and run. It is an important step because it defines the environment in which the operating system will operate.

Once the configuration was complete, the build process was started. This step converts the source code into a runnable operating system. It is one of the longest steps in the process and may take a long time depending on the computer's performance. During this stage, the system compiles many files and prepares everything needed to run Fuchsia OS.

***Finally***, after the build process was completed, Fuchsia OS was launched using an emulator. This allowed the system to run in a virtual environment, just like a real operating system. The interface of Fuchsia OS appeared on the screen, confirming that the installation was successful.

During the process, screenshots were taken at different stages to show the progress and final result. These screenshots are included in the project as evidence of successful installation.

### ***Summary of Steps :***

- Install Required Tools
- Install essential development tools such as Git, Python, and other build dependencies needed for Fuchsia OS development.



- Download Fuchsia OS Source Code
- Clone the official Fuchsia repository from Google using Git to get the operating system source files.
- Set Up the Development Environment
- Configure the environment by initializing necessary scripts and setting up system paths required for building Fuchsia.
- Configure the Build System
- Select the target build configuration (for example, x64 workstation build) to prepare the system for compilation.
- Build the Operating System
- Compile the Fuchsia OS source code using the build tools to generate system images.
- Run Fuchsia OS Using Emulator
- Launch the built system using the built-in emulator to test and run Fuchsia OS in a virtual environment.

## E. Issues (Problems Faced During Fuchsia OS Installation and Setup)

### 1. *Large Source Code Size and Slow Download :*

One of the main challenges was downloading the Fuchsia OS source code. The repository is very large, so cloning it using Git took a long time. In some cases, unstable internet connection caused interruptions, which made the process even more difficult and time-consuming. This required restarting the download and ensuring a stable network connection.

### 2. *System Performance and Hardware Requirements:*

Fuchsia OS building requires a powerful system with good CPU, sufficient RAM, and enough storage space. On average or low-performance computers, the system became slow during the build process. The compilation sometimes took several hours, and in some cases, the system lagged or became temporarily unresponsive due to high resource usage.



3. *Environment Setup Difficulties:*

Setting up the development environment was another challenge. It required careful configuration of tools, scripts, and system paths. If any step was missed or done incorrectly, it caused errors later in the process. For example, improper initialization caused commands like `fx` not to work until the environment was correctly set up again.

4. *Missing Dependencies:*

Some required software packages such as Python libraries, compilers, and build tools were not available by default. These had to be installed manually. If any dependency was missing, the build process would fail, and error messages would appear. This required troubleshooting and installing the correct packages one by one.

5. *Build Errors During Compilation:*

While compiling the source code, several build errors were encountered. These errors were often caused by missing dependencies, incorrect configuration, or incomplete setup. Fixing these issues required checking error logs carefully and repeating the build process multiple times until successful.

6. *Emulator Performance Issues:* Running Fuchsia OS using the emulator (`fx emu`) also presented challenges. The emulator required hardware virtualization support, and without it, performance was very slow or the emulator failed to start properly. This made testing difficult on low-spec systems.

## F.Solutions (How the Problems Were Solved)

1. *Large Source Code Size and Slow Download:*

To solve this issue, a stable and high-speed internet connection was used to avoid interruptions during cloning. In some cases, the download was done in a controlled environment to ensure the process was not stopped. Also, using



Git efficiently helped resume downloads instead of starting over.

2. *System Performance and Hardware Limitations:*

To handle performance problems, unnecessary background applications were closed to free up RAM and CPU usage. In some cases, the build process was run on a system with better specifications or inside a properly configured virtual machine with increased memory and processor allocation.

3. *Environment Setup Problems:*

This issue was solved by carefully following the official Fuchsia setup documentation step by step. Missing steps were corrected by re-initializing the environment using proper scripts. Ensuring that all paths and environment variables were correctly set helped fix command errors like `fx` not found.

4. *Missing Dependencies:*

Missing packages were installed manually using terminal commands such as `apt install`. Before starting the build, all required tools (Git, Python, build tools, and libraries) were verified and installed in advance. This reduced errors during compilation.

5. *Build Errors During Compilation:*

Build errors were solved by carefully reading error messages and logs. Most issues were fixed by installing missing dependencies, correcting configuration commands, or cleaning and rebuilding the project. Running `fx clean` and rebuilding also helped resolve repeated errors.

6. *Emulator Performance Issues:* Emulator issues were solved by enabling virtualization (VT-x/AMD-V) in BIOS settings. In addition, increasing RAM and CPU allocation for the virtual machine improved performance. Using a system with hardware acceleration support also helped the emulator run more smoothly.

## G. Filesystem support

In an operating system simply means how the system stores files, organizes them, and allows users or applications to access them. In Fuchsia OS, this system is designed in a modern and smart way compared to traditional



Edit with WPS Office

operating systems like Windows or Linux. The main goal is to make file handling safe, fast, and reliable.

In Fuchsia OS, storage is not handled in a simple direct way. Instead, it uses a service-based and component approach, meaning the system manages files through controlled services. This helps improve security because applications cannot freely access or change system files unless permission is granted. This reduces the chance of system damage or unwanted changes.

Another important idea in Fuchsia is that each application is given its own separate storage space. This is often called isolated or sandboxed storage. It means one app cannot directly interfere with another app's files. For example, if one app crashes or has a bug, it will not affect other apps or system data. This makes the system more stable and safe for users.

Fuchsia also uses something called a virtual file system. This works like a middle layer between applications and the real storage hardware. Instead of apps talking directly to the disk, they go through this virtual system. This makes the operating system more flexible because it can handle different types of storage in a unified way.

The system also separates temporary data and permanent data. Temporary files are used for short-term tasks and can be removed when the system restarts, while permanent files are stored safely for long-term use. This helps the system manage memory better and improves performance.

Another strong feature of Fuchsia's filesystem is data safety. Even if the system suddenly shuts down or crashes, it is designed in a way that reduces the chance of losing important data. This makes it more reliable for users.

Security is also a big part of the design. Applications cannot access files freely; they must request permission first. This protects the system from harmful or unauthorized access.



## H.Advantages and Disadvantages of Fuchsia OS

### ★ *Advantages*

i. **Modern Architecture:**

Fuchsia OS is built with a modern design that is different from traditional operating systems. It uses a component-based structure, which makes it more flexible and easier to manage.

ii. **High Security:**

It has a strong security model where applications cannot freely access system files. Each app runs in a restricted environment (sandbox), which protects the system from viruses and unauthorized access.

iii. **Good Performance Design:**

Fuchsia is designed to handle tasks efficiently, even when multiple applications are running. Its lightweight and optimized structure helps improve overall system performance.

iv. **Stability and Reliability:**

Because of apps are isolated from each other, a failure in one application does not affect the whole system. This makes the system more stable.

v. **Future-Oriented System:**

Fuchsia OS is designed for modern devices such as IoT devices, smart systems, and mobile platforms, making it a future-ready operating system.

vi. **Flexible File and System Management:** It uses virtual file systems and modular components, which makes it easier to manage different types of storage and system services.



## *Disadvantages*

- i. **Still Under Development:** Fuchsia OS is not a fully mature operating system yet. Many features are still being developed, so it is not widely used in real-world environments.
- ii. **Limited Software Support:** It does not support most common applications like Windows or Linux programs, which limits its usability for everyday users.
- iii. **Difficult Installation Process:** Installing and building Fuchsia OS is complex and requires technical knowledge, making it difficult for beginners.
- iv. **High System Requirements:** It needs a powerful system with good hardware resources for building and running smoothly, especially when using an emulator.
- v. **Lack of User Interface Familiarity:** Since it is different from traditional operating systems, users may find it confusing at first.

## **I. Conclusion**

In conclusion, Fuchsia OS is a modern and innovative operating system that represents a new direction in operating system design. Unlike traditional systems such as Windows or Linux, Fuchsia is built with a completely different architecture based on components, modular services, and strong security principles. This makes it more flexible, more secure, and better suited for future technologies.

Through this project, it was possible to understand how Fuchsia OS is downloaded, configured, built, and run using an emulator. Even though the process was not simple, it provided a very good learning experience. The installation required several steps such as setting up the environment, installing dependencies, cloning the source code, and building the system. Each of these steps helped in understanding how a real



operating system is developed from scratch.

During the work, different challenges were also faced, such as slow downloading due to large source files, system performance issues, missing dependencies, and build errors. However, solving these problems improved practical skills in troubleshooting, command-line usage, and Linux-based system management. It also gave a better understanding of how complex operating system development can be in real-world scenarios.

One of the most interesting parts of this project was running Fuchsia OS using an emulator. This helped to see how the system works in a virtual environment without needing actual hardware. It also showed how modern operating systems are tested before being released for real devices.

However, it is important to note that Fuchsia OS is still under development. It is not yet fully ready for daily personal or business use. It has limited software support, and the installation process is still complex for beginners. These limitations show that it is still evolving and being improved by developers.

Overall, this project was very helpful in understanding modern operating system concepts and development processes. Fuchsia OS shows strong potential for the future, especially for smart devices, embedded systems, and next-generation computing. It may become more powerful and widely used in the future as development continues.

## **J. Future Outlook / Recommendation**

Fuchsia OS is still a developing operating system, but it has a very promising future. It is designed with modern technology in mind, focusing on flexibility, security, and performance. As technology continues to evolve, Fuchsia OS has the potential to become an important system for future devices such as smartphones, smart home devices, embedded systems, and IoT-based technologies.

In the future, one expected improvement is the expansion of software and application support. Currently, Fuchsia OS has limited compatibility with popular applications, but with continuous development, it may support more apps and tools, making it more useful for everyday users.



Edit with WPS Office



Another important future direction is better hardware compatibility. As more devices are designed to support Fuchsia, it could become more stable and easier to install on different systems without requiring complex configurations.

It is also expected that installation and development processes will become simpler. At the moment, building Fuchsia OS is difficult for beginners, but future updates may introduce easier setup tools and more user-friendly installation methods.

***Recommendations:***

- Developers should continue improving system stability and reducing build complexity to make Fuchsia OS easier for beginners and students.
- More documentation and tutorials should be provided so that new users can understand the installation and development process more easily.
- Improving application support is very important so that Fuchsia OS can be used for real-world daily tasks, not only for testing and development.
- Hardware manufacturers should be encouraged to support Fuchsia OS to increase its usability across different devices.
- Students and researchers are encouraged to explore Fuchsia OS because it provides valuable knowledge about modern operating system design and architecture.

Fuchsia OS may become widely used in the future, especially in smart devices and embedded systems. It is recommended for research and learning purposes rather than daily use at this stage. Continued development and community support will improve its usability.

## 2. *Virtualization in Operating System*



Edit with WPS Office

## *What is Virtualization?*

Virtualization in an operating system is a technology that allows a single physical computer to create and run multiple virtual computers (called virtual machines) at the same time. Each virtual machine behaves like a real computer with its own operating system, memory, storage, and applications, even though they all share the same physical hardware.

For example, a Windows computer can run Linux inside a virtual machine using software like VirtualBox or VMware.

## *Why Virtualization is Important?*

Virtualization is widely used in modern computing because it provides many **advantages**:

### *Efficient Use of Resources*

Instead of using one computer for one task, virtualization allows multiple systems to run on a single machine, making better use of CPU, RAM, and storage.

### *Cost Reduction*

Organizations do not need to buy many physical computers. One powerful system can run many virtual systems.

### *Testing and Development*

Developers can test software on different operating systems without needing separate physical devices. This reduces risk and improves flexibility.



## *Security and Isolation*

Each virtual machine is isolated. If one system crashes or gets infected with malware, it does not affect the main system or other virtual machines.

## *Learning and Training*

Students and beginners can safely practice installing and using different operating systems without damaging their real computer.

## *How Virtualization Works in OS?*

Virtualization works through a special software layer called a Hypervisor. The hypervisor manages and distributes hardware resources between virtual machines.

### *Main Components:*

#### *1. Physical Hardware*

- This is the actual computer system, including:
- CPU (processor)
- RAM (memory)
- Storage (hard disk/SSD)

#### *2. Hypervisor (Virtual Machine Manager)*

The hypervisor is the most important part of virtualization. It sits between hardware and virtual machines and controls resource sharing.



***There are two types:***

Type 1 (Bare-metal Hypervisor):

- Installed directly on hardware
- No need for a host operating system  
Example: VMware ESXi, Microsoft Hyper-V

Type 2 (Hosted Hypervisor):

- Installed on top of an existing operating system
- Easier to use for students and beginners
- Example: VirtualBox, VMware Workstation

### ***3. Virtual Machines (VMs)***

These are the fake computers created by the hypervisor. Each VM has:

Its own operating system (e.g., Ubuntu, Windows, etc.)

- Virtual CPU
- Virtual RAM
- Virtual storage
- Applications and files

Each VM works independently, even though they share the same physical hardware



## **Working Process of Virtualization**



Edit with WPS Office

- 1. You install virtualization software (like VirtualBox or VMware).
- 2. You create a virtual machine and allocate resources (RAM, CPU, storage).
- 3. You install an operating system inside the virtual machine.
- 4. The hypervisor manages resource sharing between the main system and virtual machines.
- 5. You can run multiple operating systems at the same time on one computer.



## Conclusion

Virtualization in an operating system is a powerful technology that allows one physical computer to run multiple virtual systems efficiently. It improves resource utilization, reduces cost, increases security, and supports testing and learning. The hypervisor plays a key role by managing virtual machines and ensuring smooth performance.

